Apache Spark
Structured Streaming

Kodjo Klouvi | kodjo.klouvi@gmail.com

1- Introduction to Structured Streaming

What is Data Streaming?

- Data is generated continuously (Logs, sensors, financial data, etc.)
- Streaming = processing data as it arrives, not after

Streaming vs Batch Processing

Feature	Batch Processing	Streaming Processing
Input Data	Bounded (static)	Unbounded (infinite)
Latency	High	Low
Processing	Periodic	Continuous or micro-batch
Use Case Example	Daily reports	Real-time dashboards

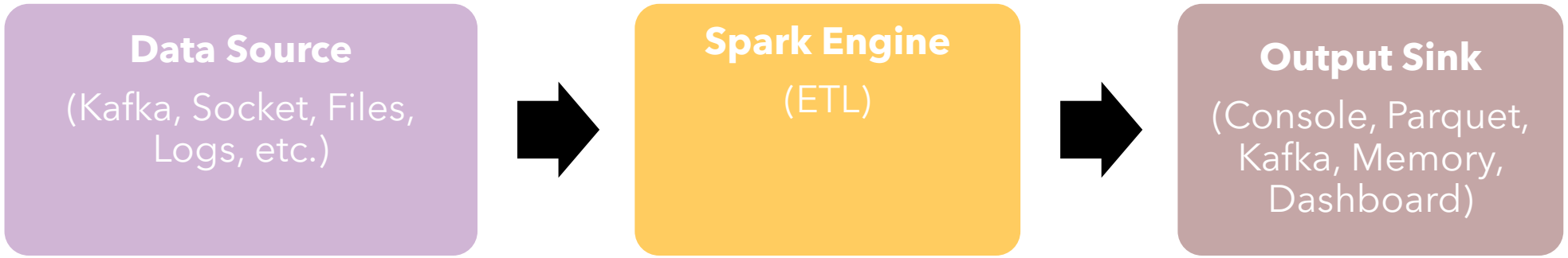
What is Structured Streaming?

- A high-level Streaming API built on Spark Core and Spark SQL
- Treats streaming data a continuous (unbounded) table
- Uses familiar DataFrame/Dataset API
- Support SQL queries, aggregations, joins, windows, etc.

Why Structured Streaming?

- Unified programming model: same code for batch & streaming applications
- Declarative: just write queries (tell Spark what to do), Spark handles execution
- Scalable: leverage Spark's distributed engine
- Fault-tolerant: automatic recovery, state management

Structured Streaming Architecture



Key Terms

- **Input Source**: Streaming data source (Kafka, files, logs, socket, etc.)
- **Output Sink**: Where results are written (console, storage, dashboard, etc.)
- **Trigger**: Define when to process data
- **Watermark**: Helps manage late data
- **State**: Used for aggregations and joins over time (*event-time*)

2- Structured Streaming – Core Concepts

- Common and unified API for batch and stream processing
- Use of SQL queries to process streaming data
- Cover various of stream processing use cases

Input Sources

- Structured Streaming supports multiple sources of real-time data
  - **Kafka**: real-time messaging system (most used)
  - **Socket**: simple line-based text stream (network)
  - **File**: watches folders for new files (file systems, FTP, etc.)
  - **Cloud sources**: Amazon S3, Google Cloud Storage (file)
- Input must be append-only and schema-compatible

DataFrames & Datasets

- Structured Streaming uses **DataFrame/Dataset APIs**:
  - Unified APIs for both **batch and streaming**
  - Streaming queries = **incremental** queries on unbounded Dataframes
  - Can **use Spark SQL, Transformations, UDFs**

Static Schema over Dynamic Data

- Streaming data can evolve, but Spark requires a static schema
- Schema is defined at the start of the streaming query
- This enables:
  - Optimized query planning
  - Serialization/deserialization
- Best practice: define explicit schema instead of inferring

The Unbounded Table Model

- Structured Streaming treats streaming data as a **logical table**:
  - A growing table to which new rows are continuously added
- This allows:
  - Declarative queries (over time)
  - Reuse of SQL concepts (filters, joins, aggregation, etc.)

Micro-Batches
(Default Execution Mode)

- Spark processes data in **small time intervals** (micro-batches)
- Each **micro-batch** reads new data, runs the query, and outputs results
- Example: Trigger every 5 seconds → new batch every 5s
- Scalable – Fault-tolerant – Works well with serval input sources

3- Structured Streaming APIs

Reading a Stream: *readStream*

- Use of **spar.readStream** just like **spark.read**
- Specify the input source format
  - kafka
  - Socket
  - csv, json (from directory)

```
val df = spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers",
"localhost:9092")
  .option("subscribe", "topic-name")
  .load()
```

Writing a Stream: *readStream*

- Use of **spar.writeStream** to start a streaming query
- Call ***.start()*** to launch the streaming job

```
df.writeStream
  .format("console")
  .outputMode("append")
  .start()
```

Output Modes

Mode	Description	Use case
Append	Only new rows	Simple ETL, inserts only
Update	Rows that changed	Aggregations without watermark
Complete	All results, every time	Aggregations with full state

Output Sinks

Sink	Usage
console	Debugging/Testing
parquet, json, csv	Persistenct storage
memory	Querable in Spark SQL (testing)
kafka	Push results back to kafka topic
foreach	Custom sink (any side-effect)

Use awaitTermination() to wait for the query to finish.

4- Case Study: Reading from Kafka

Why Kafka?

- Designed for high-throughput, low-latency data ingestion
- Integrates natively with Spark Structured Streaming
- Ideal for:
  - Log collection
  - Real-time analytics
  - Data pipelines
- Kafka provides:
  - Topics = data streams
  - Partitions = scalability
  - Offsets = message tracking

Connect to Kafka

- Kafka data is returned as a DataFrame with columns:
  - key, value → as binary (Array[Byte])
  - topic, partition, offset, timestamp, etc.

```
val df = spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers",
"localhost:9092")
  .option("subscribe", "my-topic")
  .load()
```

Parse Kafka Messages

- Cast and parse the input message (event)

```
import spark.implicits._

val parsedDf = df.selectExpr("CAST(value AS STRING) as json_str")
  .select(from_json($"json_str", schema).as("data"))
  .select("data.*")
```

Example: Schema for JSON

- Aggregations, filtering, windowing, etc.

```
val schema = new StructType()
  .add("userId", StringType)
  .add("action", StringType)
  .add("timestamp", TimestampType)

parsedDf.groupBy("action").count()
```

Writing to Sink

- Write the transformed data to Parquet

```
parsedDf.writeStream
  .format("parquet")
  .option("path", "/data/output/")
  .option("checkpointLocation", "/data/checkpoints/")
  .outputMode("append")
  .start()
```

Full Example

```
val rawKafka = spark.readStream
  .format("kafka")
  .option("subscribe", "clicks")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .load()

val schema = new StructType()
  .add("userId", StringType)
  .add("page", StringType)
  .add("ts", TimestampType)

val parsed = rawKafka
  .selectExpr("CAST(value AS STRING) as json")
  .select(from_json($"json", schema).as("data"))
  .select("data.*")

val query = parsed.writeStream
  .format("parquet")
  .option("path", "/output")
  .option("checkpointLocation", "/checkpoint")
  .start()

query.awaitTermination()
```

5- Hands-on Lab